

# Day 90: Vectors and Matrices for Machine Learning

## 1. Day Number

Day 90

## 2. Topic Name

### Vectors and Matrices

Today you will learn how machine learning represents groups of numbers using **vectors** and **matrices**.

These ideas connect directly to NumPy arrays.

---

## 3. Connection

You already learned NumPy arrays such as:

```
import numpy as np

values = np.array([10, 20, 30])
```

In mathematics, an array like this can represent a **vector**.

A two-dimensional NumPy array can represent a **matrix**:

```
data = np.array([
    [10, 20],
    [30, 40]
])
```

This is important because machine learning datasets are often stored as matrices.

A useful connection is:

```
1D NumPy array → vector
2D NumPy array → matrix
```

---

## 4. Important Topics

### Scalar

A **scalar** is just one number.

For example:

```
5
42
3.14
```

In machine learning:

```
house price = 5000000
```

is one scalar value.

---

## Vector

A **vector** is an ordered collection of numbers.

For example:

```
[1200, 3]
```

This could describe one house:

```
1200 → house size  
3    → number of bedrooms
```

So one vector can represent the features of one observation.

---

## Matrix

A **matrix** is a rectangular arrangement of numbers containing rows and columns.

For example:

```
[  
  [1200, 3],  
  [1500, 4],  
  [900,  2]  
]
```

This could represent three houses.

Each row represents one house.

Each column represents one feature.

---

## Rows

A **row** usually represents one observation.

For example:

```
[1200, 3]
```

could represent one house.

---

## Columns

A **column** usually represents one feature.

For example:

```
Column 1 → house size  
Column 2 → bedrooms
```

## Shape

The **shape** tells us the dimensions of an array or matrix.

For example:

```
3 rows × 2 columns
```

has shape:

```
(3, 2)
```

Think:

```
shape = (number of rows, number of columns)
```

## 5. Foundational Notes

Machine learning models usually work with numbers.

Suppose you have a dataset:

| House | Size | Bedrooms |
|-------|------|----------|
| A     | 1200 | 3        |
| B     | 1500 | 4        |
| C     | 900  | 2        |

The words **A**, **B**, and **C** are labels for us.

The numerical information can be represented as:

```
[  
  [1200, 3],  
  [1500, 4],  
  [900, 2]  
]
```

This matrix can then be stored in NumPy and eventually given to an ML model.

The key idea is:

```
rows    → observations  
columns → features
```

---

## 6. Feature Vector Using One House

Suppose one house has:

```
Size = 1200 square feet  
Bedrooms = 3
```

We can describe that house using a feature vector:

```
[1200, 3]
```

Here:

```
Feature 1 = 1200  
Feature 2 = 3
```

In NumPy:

```
house = np.array([1200, 3])
```

Its shape is conceptually:

```
(2,)
```

because it contains two values in a one-dimensional array.

For beginner-level ML thinking, the important idea is simply:

| One observation can be represented by a vector of feature values.

---

## 7. Data Matrix Using Multiple Observations

Now suppose we have three houses:

```
House A → size 1200, bedrooms 3  
House B → size 1500, bedrooms 4  
House C → size 900, bedrooms 2
```

Each house has its own feature vector:

```
House A → [1200, 3]
House B → [1500, 4]
House C → [900, 2]
```

Putting those vectors together gives a matrix:

```
[
  [1200, 3],
  [1500, 4],
  [900, 2]
]
```

Now:

```
Rows      = 3 houses
Columns   = 2 features
```

Therefore the shape is:

```
(3, 2)
```

---

## 8. Easy Example

Imagine customer data with two features:

```
age
monthly_spending
```

One customer:

```
Age = 25
Monthly spending = 4000
```

Feature vector:

```
[25, 4000]
```

Three customers:

```
[
  [25, 4000],
  [30, 5500],
  [22, 3000]
]
```

This matrix has:

```
3 rows
2 columns
```

so its shape is:

```
(3, 2)
```

In NumPy:

```
customers = np.array([
    [25, 4000],
    [30, 5500],
    [22, 3000]
])
```

You could inspect:

```
customers.shape
```

to see the dimensions.

---

## 9. Problem Statement

Represent three houses using two numerical features:

```
House A:
size = 1000
bedrooms = 2

House B:
size = 1400
bedrooms = 3

House C:
size = 1800
bedrooms = 4
```

Your task is to:

1. represent each house as a feature vector
2. combine the three vectors into one matrix
3. identify the number of rows
4. identify the number of columns
5. determine the matrix shape
6. represent the matrix using a NumPy array

Do not perform any advanced matrix calculations yet.

---

## 10. Concepts Used

You will use:

- scalar
- vector
- feature
- feature vector
- matrix
- observation
- row
- column
- dimensions
- shape
- NumPy array

A useful summary is:

|                       |          |
|-----------------------|----------|
| one number            | → scalar |
| one group of features | → vector |
| many feature vectors  | → matrix |

---

## 11. Thought Process

Start with one house.

Ask:

| How many features describe this house?

In the exercise:

|          |
|----------|
| size     |
| bedrooms |

So there are:

|            |
|------------|
| 2 features |
|------------|

That means each house's vector contains two numbers.

For example:

|           |
|-----------|
| [1000, 2] |
|-----------|

Then ask:

| How many houses are there?

There are:

3 houses

Therefore there should be:

3 vectors

Stack those vectors as rows:

```
[  
  [house A features],  
  [house B features],  
  [house C features]  
]
```

Finally count:

```
rows    = number of houses  
columns = number of features
```

That gives the matrix shape.

---

## 12. Beginner-Friendly Pseudocode / NumPy Representation

Pseudocode:

```
START  
  
create vector for house A  
  [size, bedrooms]  
  
create vector for house B  
  [size, bedrooms]  
  
create vector for house C  
  [size, bedrooms]  
  
combine all three vectors into one matrix  
  
print matrix  
  
print matrix shape  
  
END
```



NumPy structure:

```
import numpy as np

houses = np.array([
    [size_house_a, bedrooms_house_a],
    [size_house_b, bedrooms_house_b],
    [size_house_c, bedrooms_house_c]
])

print(houses)
print(houses.shape)
```

Notice the structure:

```
outer brackets → complete matrix

inner brackets → individual house vectors
```

---

## 13. Easy Edge Cases

### One Feature

Suppose each house only has:

```
size
```

Then your data might look like:

```
[
    [1000],
    [1400],
    [1800]
]
```

You still have:

```
3 observations
1 feature
```

So the matrix shape would be:

```
(3, 1)
```

---

## One Observation

Suppose you have only one house with two features:

```
size = 1000  
bedrooms = 2
```

As a matrix, you might represent it as:

```
[  
  [1000, 2]  
]
```

Now:

```
1 row  
2 columns
```

so the shape is:

```
(1, 2)
```

This is different from the one-dimensional vector:

```
[1000, 2]
```

whose NumPy shape would normally be:

```
(2,)
```

That distinction becomes useful later in machine learning.

---

## 14. Expected Matrix and Shape

For the exercise, your final structure should contain three rows:

```
House A  
House B  
House C
```

and two columns:

```
size  
bedrooms
```

So conceptually:

```
[
    [size A, bedrooms A],
    [size B, bedrooms B],
    [size C, bedrooms C]
]
```

Expected dimensions:

```
Rows      = 3
Columns   = 2
```

Therefore:

```
Shape = (3, 2)
```

Try filling in the actual values yourself.

---

## 15. Hint Only

Start by turning each house into this pattern:

```
[size, bedrooms]
```

So House A begins with:

```
[1000, 2]
```

Do the same for House B and House C.

Then place all three vectors inside another pair of brackets:

```
houses = np.array([
    [...],
    [...],
    [...],
])
```

To determine the shape, ask:

```
How many houses?
→ number of rows
```

```
How many features for each house?
→ number of columns
```

The key lesson for **Day 90** is:

**A vector can represent the features of one observation, while a matrix can represent many observations. In machine learning, rows commonly represent observations and columns commonly represent features.**